Progress Tab

🧠 LazyHC2L Developer & Team Guide

📘 Overview

🎯 Objective

The LazyHC2L tool extends the **Hierarchical Cut 2-Hop Labeling (HC2L)** algorithm to handle **dynamic and disruption-prone road networks** using two mechanisms:

- **Lazy Updates:** Defer recomputation until affected nodes are queried.
- **Threshold-Based Triggering:** Only update when disruption impact exceeds a defined severity threshold (τ).

The goal is to deliver **efficient real-time routing** that balances accuracy and computation cost while adapting to real-world road changes (traffic, closures, accidents).

🧩 System Architecture Summary

🔧 Current Architecture Components (from Thesis)

1. **Dataset Input** – imports Quezon City OSM road network + HERE Traffic Flow data.
2. **Network Change Detector** – identifies disrupted edges.
3. **Impact Evaluator** – computes ImpactScore = f(Δw) × f(jam) × f(closure)
4. **Threshold Decision Module** – compares ImpactScore ≥ τ to decide update type.
5. **Lazy Update Mode** – marks nodes as dirty, updates on query.
6. **Immediate Update Mode** – recomputes all affected labels instantly.
7. **Query Engine** – processes source-destination requests, handles lazy repair.

8. **Frontend (Flask + JS)** – displays route, traffic, disruptions, and comparisons.

🧠 Algorithm Summary

Component	Description
HC2L	Static hierarchical labeling algorithm with fast query and small label size.
DHL (Baseline)	Dynamic labeling using dual hierarchies for query/update.
LazyHC2L (Proposed)	Adaptive HC2L with lazy and threshold-based label repair.

Lazy Update Logic

- Labels are **not recomputed immediately** when a disruption occurs.
- Affected nodes are marked **dirty** until a query touches them.
- When queried, the algorithm **repairs** only the relevant subgraph.

Threshold Triggering Logic

- Computes disruption severity via **ImpactScore**.
- If `ImpactScore ≥ τ` → immediate recompute.
- If `ImpactScore < τ` → lazy mark only.

🧠 PANEL FEEDBACK → ACTION PLAN

Panel Concern	Required Fix / Feature	Implementation Area

Real-world adaptation	Integrate real APIs (MMDA, Waze, HERE); simulate disruptions dynamically	Backend & Simulation
Algorithm clarity	Highlight that HC2L is *labeling-based*, not a shortest path algorithm	UI & Docs
Performance stress test	Add performance/memory dashboard	Backend & UI
Visualization gap	Visualize DHL vs LazyHC2L updates on map	Frontend (Map overlay)
Simulation realism	Add time-based traffic profiles, rerouting logic	Simulation Layer
User input	Allow search-based locations, not only pin	Frontend (Geolocation)
Accuracy validation	Compare with Google Maps (Fréchet + Overlap)	Evaluation Module
Memory optimization	Add LRU caching, label pruning	Backend Core
One-way roads	Enforce OSM directionality	Graph Preprocessing

🧭 USER INTERFACE DEVELOPMENT TASKS

A. Input & Interaction

Component	Description	Status	Action
🔍 **Search Bar**	Text-based location search (Nominatim / Geopy)	❌	Add

📍 **Smart Pinning**	Pins snap to nearest road	⚠️	Fix
🚧 **Disruption Reporter**	User can mark closure/accident	✅	Improve with type icons
❌ **Unreport / Delete**	Remove disruption pins	❌	Add
🔁 **Reroute Button**	Recalculate after disruption	❌	Add
⏰ **Time Profile Selector**	Morning / Noon / Evening	❌	Add

B. Map & Visualization

Component	Function	Action
🗺 **Path Lines**	Color-coded routes (LazyHC2L, DHL, Google)	Polish colors
🟢🟡🔴 **Traffic Overlay**	Visualize congestion level	Add Layer
🟧🟩 **Update Region Overlay**	DHL vs LazyHC2L recomputed areas	Add via Leaflet polygons
📘 **Legend Panel**	Standardized colors (Blue=Lazy, Purple=DHL)	Add
📃 **Route Info Panel**	Show ETA, label count, algorithm	Enhance
📩 **Export Results**	Save JSON/CSV logs for thesis appendix	Add

C. Legend & Dashboard

Component	Description	Status

📊 **Comparison Dashboard**	Side-by-side DHL vs LazyHC2L	Add
📈 **Metrics Chart**	Label time, size, query response	Add
🔢 **Live Log Panel**	Per-query debug logs	Add
💉 **SOP Mode Toggle**	Strict mode for thesis data capture	Optional

D. Simulation Controls

Feature	Description	Action
▶️ **Run Simulation**	Start scenario (e.g., Scenario 1–5)	Update flow
🛑 **Reset Simulation**	Clear all pins/routes	Improve
📂 **Load Scenarios**	Predefined or random disruptions	Add
⏱️ **Live/Step Toggle**	Replay algorithm updates	Optional

🧠 DEVELOPER / BACKEND TASKS

Core Algorithm Enhancements

1. **Lazy Update System**
    - Add `dirty_labels` set and `mark_as_dirty()` logic.
    - Implement `lazy_update()` trigger on query.
    - Log: `ImpactScore`, `UpdateType`, and node IDs affected.
2. **Threshold Logic Integration**

- Compute `ImpactScore = fΔw × fjam × fclosure`.
- Compare to τ and trigger Lazy or Immediate Update.
- Allow τ slider in frontend for demo.

3. **Disruption Handling API**
   - `/add_disruption`, `/clear_disruption`, `/reroute` endpoints.
   - Handle static + simulated inputs.
4. **Performance Optimization**
   - Add **LRU cache** for reused paths.
   - Implement **garbage collection** for stale labels.
   - Record memory/time logs.
5. **Graph Directionality Fix**
   - Use `network_type="drive"` in OSMnx to enforce one-way.
   - Validate edges using `oneway` tag.
6. **Logging & Visualization Output**
   - Output updated node lists for both DHL and LazyHC2L.

Return JSON to frontend:

```
 {
  "disruption_segment": [104,105],
  "dhl_updated_nodes": [1,2,3,4,5],
  "lazy_updated_nodes": [3,4]
}
```

-

🧪 SIMULATION & TESTING REQUIREMENTS

Test Scenarios

Scenario	Description
1	Light congestion only
2	Road closure
3	Mixed (accident + congestion)
4	Randomized disruptions
5	Historical rush-hour simulation

Metrics to Log

- Labeling time (s) - How long it takes to build the label index for the graph using HC2L or its variants. This includes preprocessing steps like graph partitioning, label construction, and pruning.
    - Used in: SOP 1 & 2 – System efficiency
    - Interpreted as: Lower = faster system initialization

- Labeling size (MB) - Total memory size of the constructed labels after preprocessing. Represents the storage required for all node labels in the index.
    - Used in: SOP 1 & 2
    - Interpreted as: Lower = more efficient memory use

- Query response (µs) - Time taken to compute a single shortest path between an origin and destination using the label index. For LazyHC2L, this includes any triggered lazy update time.
    - Used in: SOP 1 & 2
    - Interpreted as: Lower = faster pathfinding at runtime

- Fréchet distance (m) - Measures the geometric similarity between two paths (e.g., LazyHC2L vs Google Maps). Lower Fréchet values mean the paths are spatially closer in shape and alignment.
    - Used in: SOP 3 – Accuracy

    - Interpreted as: Lower = route is more similar to reference

- Segment overlap (%) - Percentage of road segments shared between the tool-generated route and the reference route (Google Maps). Calculated by matching IDs or spatial proximity of segments.
    - Used in: SOP 3
    - Interpreted as: Higher = route is closer to reference path

- Nodes updated per algorithm - Total number of nodes that had their labels recomputed after a disruption.
- For DHL: updates are immediate and broad
- For LazyHC2L: updates are lazy and limited (triggered on query)
    - Used in: SOP 2 – System responsiveness
    - Interpreted as: Lower = more efficient adaptation

- Memory used (MB) - Total runtime memory consumed during simulation, including label storage, dirty flag states, and temporary buffers.
    - Used in: SOP 1 + optimization testing
    - Interpreted as: Lower = better scalability

✅ Optional Formatting Tip (for Tooltips or UI Panels)

Metric	Tooltip
Labeling Time	"Time to construct all node labels before queries"
Labeling Size	"Total memory occupied by all precomputed labels"
Query Response	"Time taken to compute shortest path using labels"
Fréchet Distance	"Geometric difference between tool route and reference"

Segment Overlap	"How many road segments match the reference path"
Nodes Updated	"Number of nodes reprocessed after disruption"
Memory Used	"Overall memory usage during simulation run"

Output Format

CSV / JSON export for each trial (for SOP evaluation). Include:

trial,algorithm,label_time,label_size,query_time,frechet,overlap,updated_nodes

---

📦 DELIVERABLES FOR PROGRAMMER

Deliverable	Description
✅ **Working Backend**	Fully functional LazyHC2L + DHL comparison engine
✅ **UI Dashboard**	User + Developer views with overlays & metrics
✅ **Simulation Suite**	Scenario loader, rerouting demo
✅ **Export System**	JSON/CSV logs for thesis results
✅ **Documentation**	Inline code comments + API endpoints list

---

📚 Suggested Dev Tools & Frameworks

- **Backend:** C++, Flask (Python), SQLite, NumPy, Pandas
- **Frontend:** Leaflet.js / Mapbox GL JS, Bootstrap / Material UI
- **Visualization:** Chart.js / Plotly for dashboard
- **Testing:** pytest (Python), CSV output validation

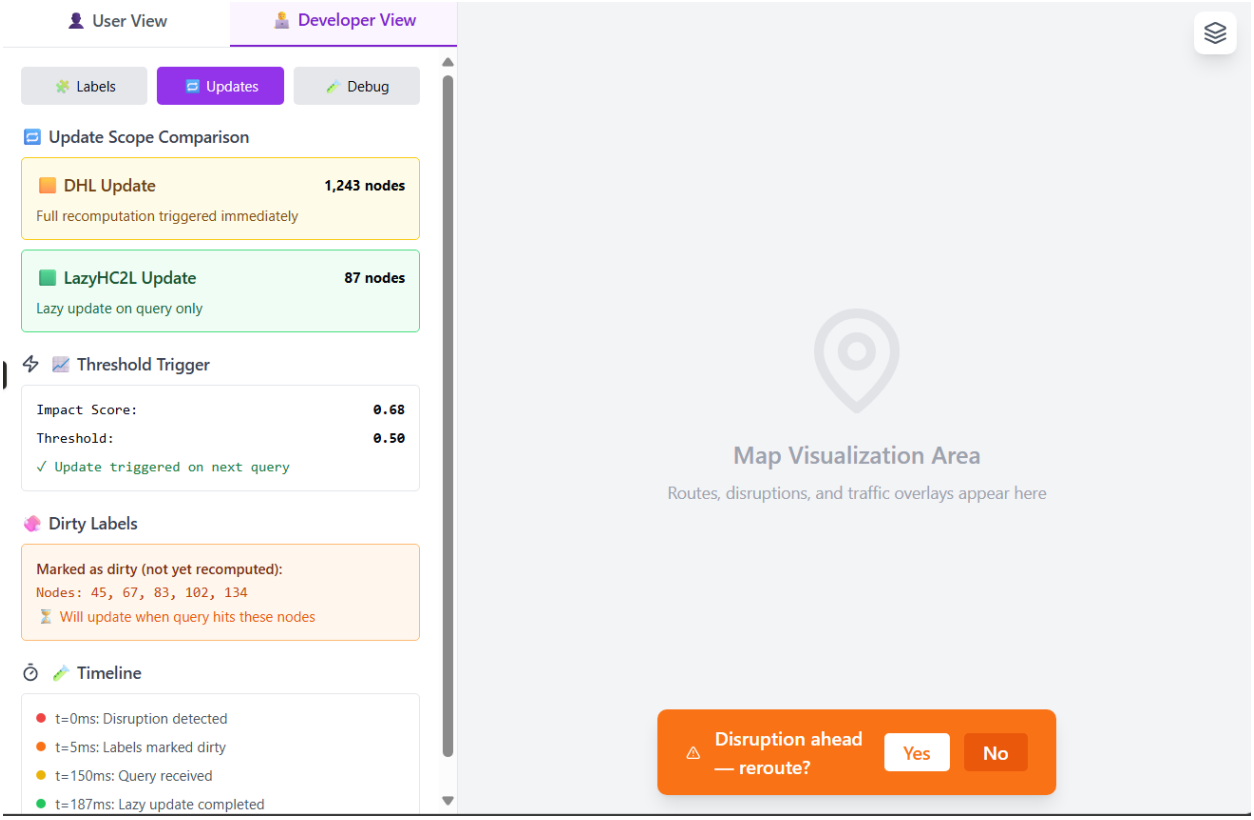
🧭 Implementation Flow Summary

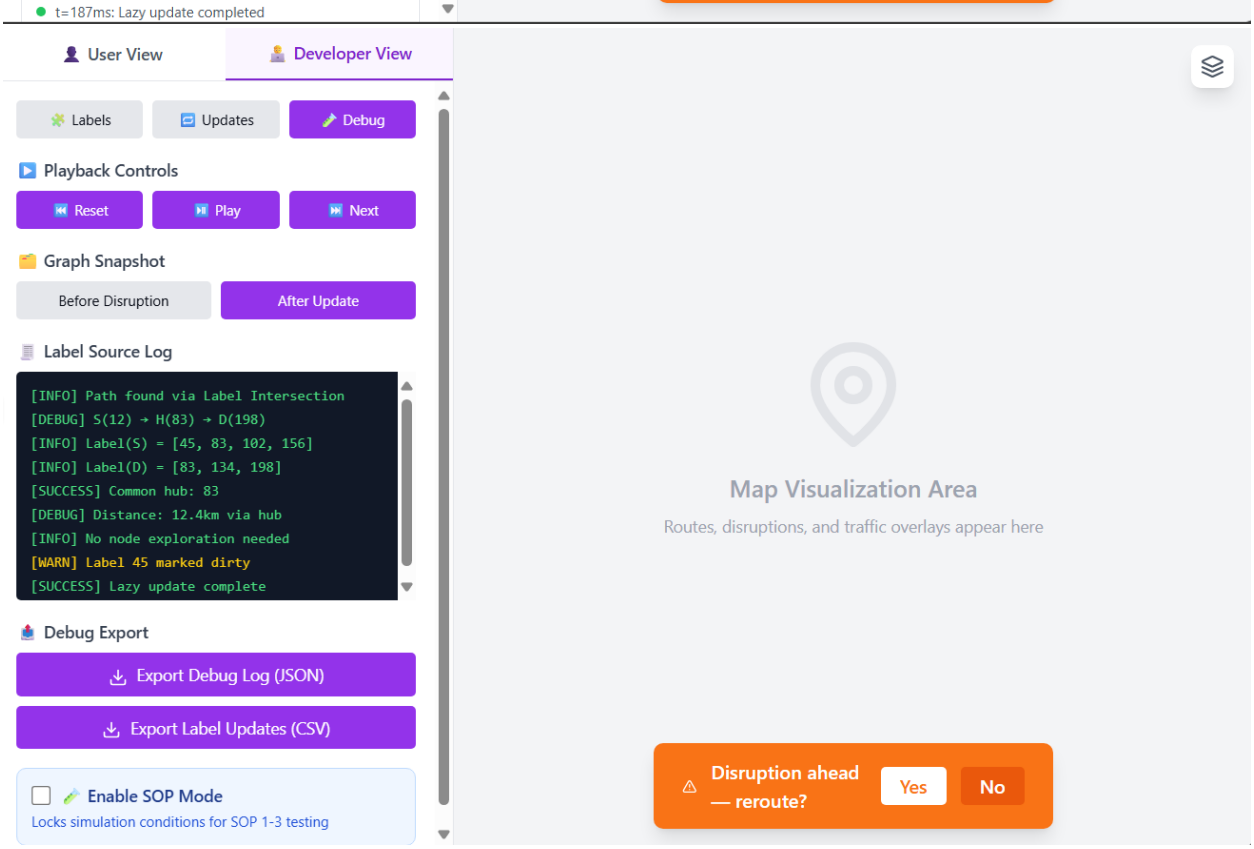
1. Load base map (Quezon City, OSMnx)
2. Load disruptions (HERE or synthetic)
3. Run DHL and LazyHC2L
4. Log updated nodes & query times
5. Display overlay comparison
6. Export results (for SOPs)

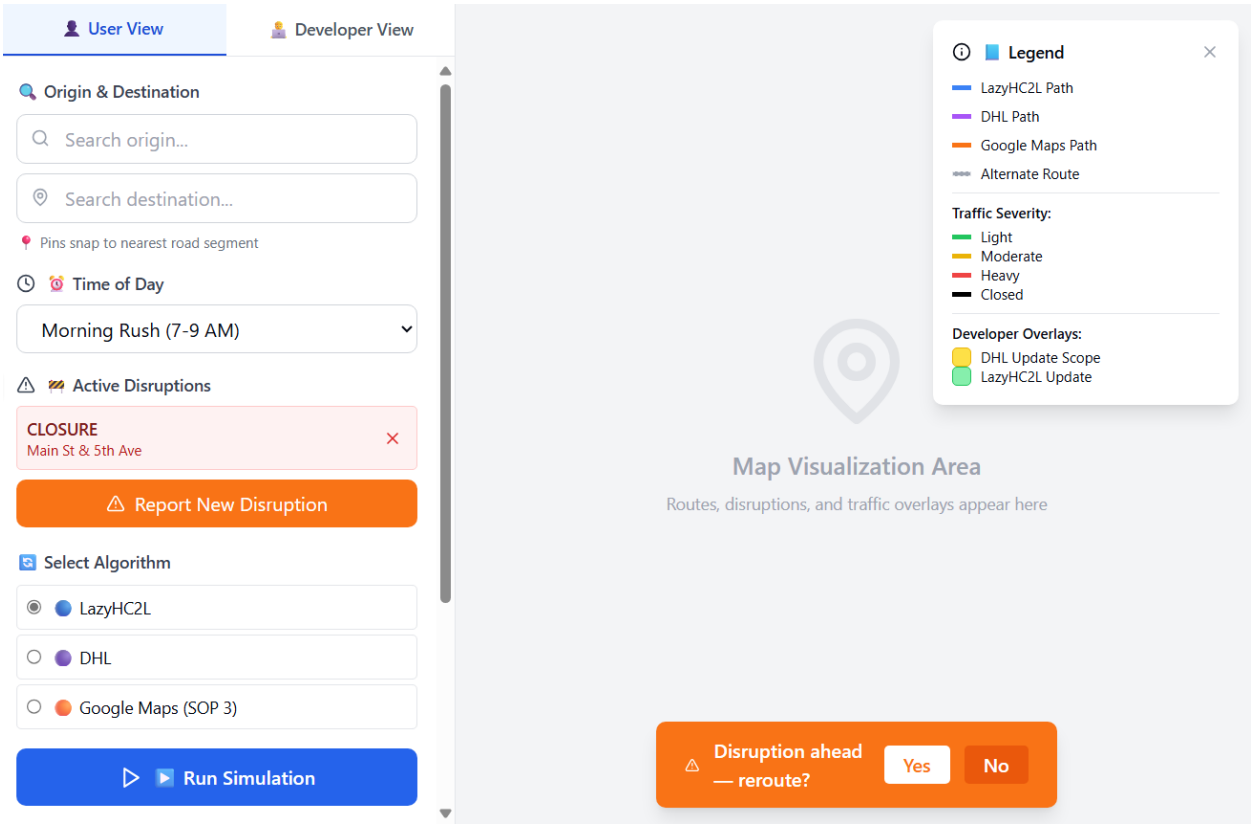
✅ Success Criteria

- Clear visual proof of LazyHC2L's adaptive update vs DHL.
- Correct handling of one-way and restricted roads.
- Consistent comparison results with Google Maps.
- Logged metrics usable for SOP 1–3 analysis.
- Frontend simulation runs smoothly and interactively.

SAMPLE UI ONLY






User View
Developer View
Origin & Destination
Search origin...
Search destination...
Pins snap to nearest road segment
Time of Day
Morning Rush (7-9 AM)
Active Disruptions
CLOSURE
Main St & 5th Ave
Report New Disruption
Select Algorithm
LazyHC2L
DHL
Google Maps (SOP 3)
Run Simulation
Legend
LazyHC2L Path
DHL Path
Google Maps Path
Alternate Route
Traffic Severity:
Light
Moderate
Heavy
Closed
Developer Overlays:
DHL Update Scope
LazyHC2L Update
Map Visualization Area
Routes, disruptions, and traffic overlays appear here
Disruption ahead — reroute?
Yes
No

Thesis

Chapter 1

THE PROBLEM AND ITS SETTING

Introduction

Real-time navigation in dynamic road networks presents a significant challenge due to the unpredictable nature of traffic conditions, road closures, and other disruptions. Classic static routing algorithms like Dijkstra's and A* provide basic implementations for shortest-path computation but are ineffective in coping with real-life situations where road conditions keep changing quickly (Zhou et al., 2024; Katre & Thakare, 2017). These algorithms are computationally intensive and require frequent re-evaluation to remain accurate, making them inefficient in large or dynamically changing networks.

To address these limitations, labeling-based algorithms, particularly Hub Labeling (HL), have been developed to allow near-instantaneous query responses through extensive precomputation. Variants like Pruned Highway Labeling (PHL) and Hierarchical 2-Hop Index (H2H) enhance query performance by narrowing the space to be searched through the use of hierarchical constructs (Akiba et al., 2014; Ouyang et al., 2018). Of these, Hierarchical Cut 2-Hop Labeling (HC2L) has been one of the most scalable and effective approaches in static environments, providing smaller label sizes, quick query times, and outstanding scalability through balanced graph partitioning and Lowest Common Ancestor (LCA) queries (Farhan et al., 2023).

Despite HC2L's benefits, it is still limited in dynamic environments. The core research issue this investigation deals with is the failure of existing HL-based systems such as HC2L to handle real-time network changes efficiently without high recomputation overhead or intolerable query delay. Even small disturbances like traffic congestion or road closures in dynamic networks

such as urban road networks greatly impact travel time. HC2L's use of static, precomputed labels becomes problematic under such
circumstances, since constant updates involve label reconstruction over large subgraphs and incur memory overhead as well as latency.

More recent dynamic adaptations like Dual-Hierarchy Labeling (DHL) have been suggested to address such issues by preserving distinct query and update hierarchies that localize the effects of changes in the weights of edges (Farhan et al., 2025). Although DHL shows substantial improvement in performance compared to earlier approaches, it adds more complexity and resources that can put a cap on its usability in environments with limited infrastructure or repeated minor interruptions. Likewise, Stable Tree Labeling (STL) enhances update efficiency through the separation of hierarchical structure from edge weights, facilitating localized intra-subgraph updates (Koehler et al., 2025). Yet STL implementations to date are mostly tested on undirected graphs and do not have strong support for directed or highly dynamic urban networks. Other dynamic algorithms, such as Partitioned Dynamic Hub Labeling (ParDHL) and HL variants accelerated through the use of GPUs, are still not practical for wide-scale real-world applications because they require high-end hardware or a significant maintenance burden (Li et al., 2024; Zhang et al., 2025).

Incremental update strategies have also gained attention due to their capability to partially recompute only the impacted regions of the network (Zhou et al., 2021). These methods minimize processing time over full recomputation but tend to add more algorithmic complexity and data management problems, which affect maintainability and consistency in large-scale systems.

In order to solve these algorithmic problems (i.e., high recomputation, incremental update inefficiency, and scalability problems in dynamic networks) this work presents a dynamic extension of HC2L. Specifically, the approach introduces two complementary mechanisms: Lazy Updates and Threshold-Based Triggering. Lazy Updates postpone label recomputation until

required, e.g., with a query involving an affected node, to avoid unnecessary computation for small updates (Aioanei, 2012). Threshold-Based Triggering triggers label updates only when disturbances surpass a specified level of severity (Stan et al., 2023), enabling effective adaptation to dynamic conditions.

This research examines the proposed Dynamic HC2L against DHL as a dynamic baseline and measures performance in a simulated model of Quezon City's dynamic traffic environment. As one of Metro Manila's most congested cities, Quezon City offers a representative test case for disruption-sensitive routing, with perpetual congestion and regular infrastructure interventions (Mangahas & Medes, 2019). In doing so, this research aims to bridge the gap between static efficiency and dynamic adaptability, contributing to scalable, practical solutions for real-time intelligent transportation systems.

Theoretical Framework

Figure 1. Theoretical Framework of Hierarchical Cut 2-Hop Labelling (HC2L)

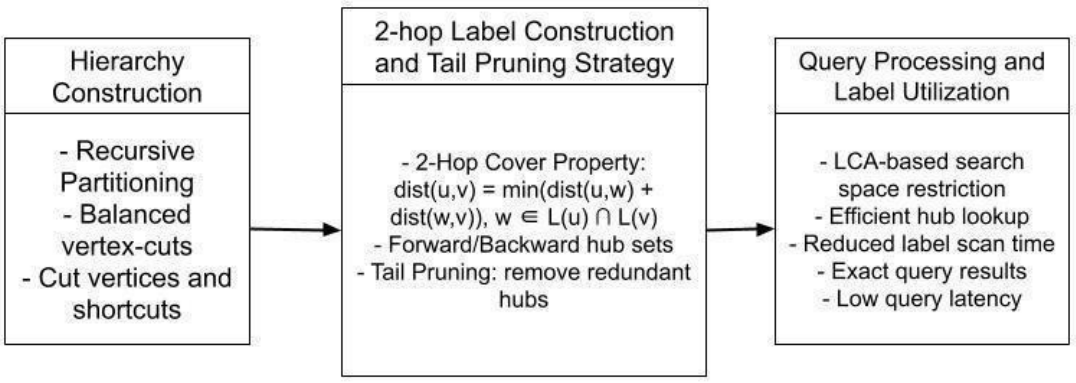


Figure 1 illustrates the structural elements of the Hierarchical Cut 2-Hop Labeling (HC2L) algorithm, a distance labeling scheme proposed by Farhan et al. (2023) to speed up shortest-path queries on vast road networks. The theoretical foundation of HC2L is built upon 2-Hop Labeling Theory (Abraham et al., 2012) and Hierarchical Graph Decomposition (Ouyang et al., 2018), both widely studied in algorithmic graph theory for enabling efficient shortest-path computation. HC2L applies hierarchical vertex cuts to recursively partition the graph into

balanced subgraphs and then builds 2-hop hub labels over the tree hierarchy therefrom. In this way, HC2L can simultaneously ensure correctness and query resolution efficiency and label size, and memory usage.

Essentially, HC2L employs hierarchical vertex-cut partitioning, recursively dividing a graph into balanced subgraphs for scalability and computational efficiency. Decomposition forms a binary tree structure where internal nodes are boundary vertex cuts and leaf nodes are atomic subgraphs. Balance is maintained by ensuring that each partition contains approximately half of the graph's vertices, minimizing edge cuts, and ensuring shallow tree depth for efficient querying.

To guarantee shortest-path accuracy among decomposed subgraphs, shortcut edges are inserted during decomposition. Artificial edges contain precomputed distances at cut vertices, eliminating complete traversal and significantly reducing query latency. HC2L maintains inter-subgraph connectivity via shortcuts without requiring additional storage, a significant improvement over existing graph decomposition methods.

Label construction in HC2L follows the 2-Hop Cover Property, where each node retains a pair of forward and backward hub node labels and distances. For any pair of nodes (u,v), the shortest path is computed via a common hub node *w* such that: $dist(u, v) = min \{ dist(u, w) + dist(w, v) \}$ for *w* where $L(u) \cap L(v)$. This computation is localized even more: w is restricted to hubs in the cut set of the Lowest Common Ancestor (LCA) of nodes u and v in the binary cut tree (Farhan et al., 2023). This restriction greatly reduces label comparisons and improves query speed without compromising shortest path correctness.

For additional label efficiency optimization, HC2L supports tail pruning, a compression method that eliminates redundant hubs overlapped by more frequent or closer labels. This compresses label sizes by 2–4× over current labeling schemes like H2H and PHL without sacrificing accuracy or query throughput.

For query resolution, HC2L reduces computation overhead by computing the Lowest Common Ancestor (LCA) of two query nodes in its hierarchical representation.
Since the LCA is on the shortest path between any two nodes, only the hub labels of the LCA and its subtrees are searched, significantly reducing retrieval complexity in large static networks.

However, HC2L's reliance on static edge weights limits its application in dynamic networks where road conditions change due to congestion, construction, or accidents. Since the label structure is built upon a precomputed static hierarchy and distances, any edge weight update typically results in complete recomputation, hence, HC2L is not feasible for real-time or highly dynamic networks. This paper fills that theoretical gap by presenting an extension that enables dynamic HC2L behavior.

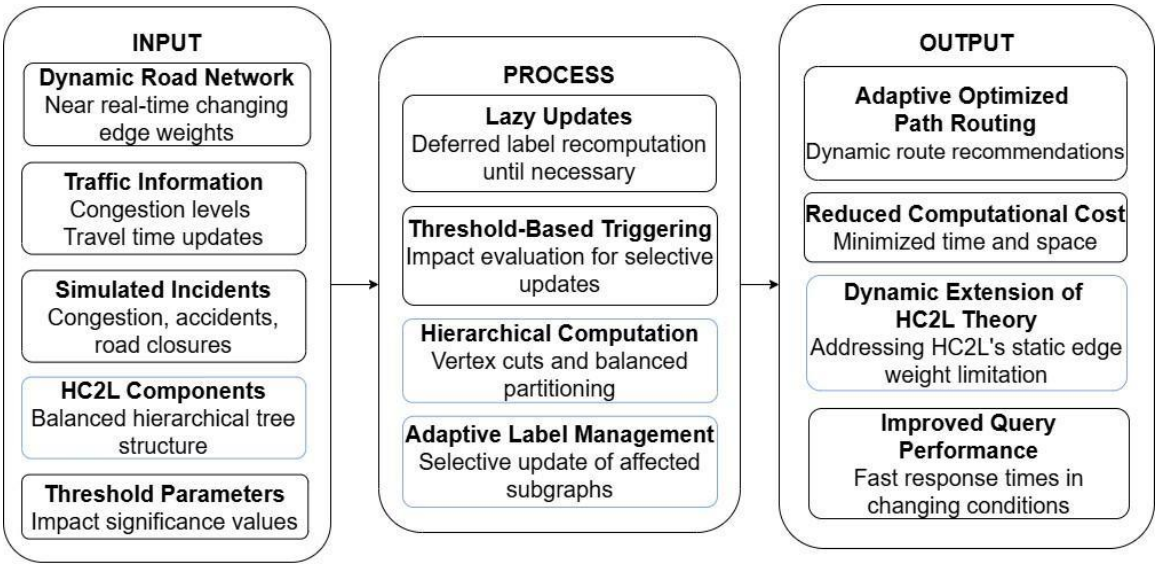
This study incorporates two theoretically grounded techniques to address the limitations of HC2L: Lazy Updates and Threshold-Based Triggering. The Lazy Updates mechanism is grounded in semidynamic shortest path computation theories, particularly those articulated in the LazyDijkDec and LazyDynDijk algorithms for dynamic graphs in a study conducted by Aioanei (2012). Rather than recalculating all cluster-level metrics like centroids, variance, and constraint violation counts on each minor structural change, Lazy
Updates defer recomputation until a query explicitly requires accurate data. This aligns with the invariance principle in dynamic path trees, where overestimated values can be maintained safely and corrected only when necessary. Such deferral exploits amortized cost efficiency by reducing redundant updates in high-frequency data mutation scenarios. Formally, given a graph $G = (V, E, w)$ where V is the set of vertices, E the set of edges, and w denotes non-negative edge weights, updates such *as* $w(u, v) \rightarrow w'(u, v)$ are not immediately propagated. Instead, a consistency invariant $I(v)$ is maintained for each vertex v, indicating whether the associated hub labels $L(v)$ are consistent with the current graph state. When a query $q(u, v)$ is issued and either $I(u)$ or $I(v)$ is violated, a localized recomputation is triggered, typically scoped to the affected subgraph or

hierarchy level. This on-demand update is coordinated through a priority queue Q, which manages stale clusters based on an error metric or query frequency. Invariants are preserved: (1) $c(h(e), T_s) \geq c(t(e), T_s) + w(e)$ for all edges e in the subtree T_s rooted at source s, and (2) a shortest-path tree S_s exists where all correct estimates match T_s, and incorrect ones have parents in Q. Edge weight decreases are handled by relaxing affected edges and propagating updates selectively (LazyDijkDec), while increases invalidate only directly impacted paths, deferring full revalidation to query time (LazyDijkInc). In practice, Lazy Updates minimize unnecessary traversals or recomputations by dynamically restoring correctness only when triggered by a read or merge operation, effectively balancing accuracy and performance under dynamic conditions.

Threshold-Based Triggering, on the other hand, is inspired by control flow gating and congestion thresholding mechanisms used in adaptive vehicular routing systems by Stan et al. (2023). It is a congestion-aware control mechanism that activates route recomputation only when vehicular density on a road segment exceeds a predefined threshold. Formally, given a graph $G = (V, E, w)$, each segment's density $\rho\_segment(t)$ is computed based on vehicle count, segment length, and lane count. When $\rho\_segment(t) > \theta$, where θ is a tunable density threshold, penalization functions adjust the segment's travel cost, speed, and delay using calibrated mappings. These functions $E\_segment(\rho)$, $S\_segment(\rho)$, and $D\_segment(\rho)$ dynamically influence routing decisions. This mechanism filters out low-impact fluctuations, improves computational efficiency, and ensures that only meaningful congestion triggers route updates.

Conceptual Framework

Figure 2. Conceptual Framework of the Study



The conceptual framework of our study is grounded in the Foundational Theory (HC2L) by Farhan et al. (2023), which provides the hierarchical structure and 2-hop labeling approach that our dynamic extensions build upon. As illustrated in Figure 2, the essential inputs for our system include the Dynamic Road Network, capturing near realtime changes in the network, and critical Disruption Events that trigger adaptive route calculations.

The proposed dynamic approach builds upon the core components of the original HC2L algorithm, particularly its hierarchical tree structure formed through balanced partitioning and vertex cuts. This structure supports efficient 2-hop labeling, enabling fast optimized path queries. HC2L is known for its scalability and compact labels, but its primary limitation lies in its static design assumptions, which our Dynamic Extensions seek to address.

These layered enhancements to the static HC2L framework introduce adaptability to dynamic road networks. Our extensions incorporate three interconnected mechanisms: Lazy Updates, Threshold-Based Triggering, and Adaptive Label Management, all working together to maintain fast query times while efficiently handling near real-time disruptions without full recomputation. These mechanisms are governed by carefully calibrated Threshold Parameters that balance computational efficiency with routing accuracy.

These configurable values determine when a network change is significant enough to trigger an update. These parameters define the sensitivity of our system to network changes and directly inform the decision-making process in our Threshold-Based Triggering mechanism, ensuring optimal balance between computational efficiency and routing accuracy.

This decision-making process evaluates the impact of network changes against the predefined Threshold Parameters. Updates are triggered exactly when network changes meet or exceed these predefined conditions, ensuring that updates occur only when a significant impact on optimal routes is detected. This mechanism works in conjunction with Lazy Updates to prevent unnecessary recalculations for minor
fluctuations.

This mechanism defers the recomputation of labels until absolutely necessary. Instead of recalculating all labels after every minor network change (which would be computationally expensive), lazy updates postpone recalculation until a query requires that information. This reduces computational overhead while maintaining accuracy when needed and complements the efficiency goals of Adaptive Label Management.

Leveraging HC2L's original strength from the foundational theory, this process uses vertex cuts and balanced partitioning to efficiently organize the network into a hierarchical structure. This hierarchical approach enables faster query processing by reducing the search space and provides the structural foundation that makes Adaptive Label Management possible.

This process selectively updates only the affected subgraphs rather than recalculating the entire network's labels. Upon detection of a significant network change (as determined by Threshold-Based Triggering), the system identifies the specific subgraphs impacted due to the hierarchical structure established through Hierarchical Computation. By focusing the update

process solely on these affected portions, the computational cost of adapting to dynamic conditions is minimized while maintaining the integrity of the Foundational Theory (HC2L).

The primary outputs of this framework are Adaptive Optimized Path Routing, delivering up-to-date route recommendations, and the Dynamic Extension of HC2L Theory, which broadens the applicability of the original static theory to dynamic road network scenarios.

Statement of the Problem

Labeling-based algorithms such as the Hierarchical Cut 2-Hop Labeling (HC2L) rely on static, precomputed labels to achieve fast query performance and compact label size in static road networks. While effective in such environments, this reliance becomes a fundamental challenge when applied to dynamic road networks. Any change in road conditions—such as congestion, accidents, closures, or user-reported disruptions— requires full-label reconstruction, resulting in high computational overhead and delayed responses. Therefore, navigation systems require algorithms that can adapt quickly to these evolving conditions without incurring excessive computation or storage costs.

Recent dynamic algorithms such as Dual-Hierarchy Labeling (DHL) address this limitation by introducing separate update and query hierarchies, allowing efficient edge weight updates without recomputing the entire label index. DHL achieves strong performance and reduced memory usage compared to earlier methods. However, it also introduces added complexity, memory overhead from maintaining two hierarchies, and potential inefficiencies under frequent minor disruptions.

Given DHL’s position as a state-of-the-art dynamic labeling approach, it serves as an appropriate and rigorous baseline for evaluating new methods. This study proposes a Dynamic HC2L algorithm that integrates lazy updates and threshold-based triggering to selectively update labels. The goal is to retain HC2L’s simplicity and high query performance while adding the ability

to handle real-time disruptions with lower structural and computational overhead than DHL. The study is guided by the following research questions:

1. What is the performance of the proposed Dynamic HC2L algorithm in terms of:
    a. Labeling time
    b. Labeling size
    c. Query response time
2. What is the difference between the proposed Dynamic HC2L algorithm and the DHL algorithm in terms of:
    a. Labeling time
    b. Labeling size
    c. Query response time
3. How similar are the paths generated by the Dynamic HC2L algorithm to real-world navigation routes generated by Google Maps in Quezon City, as measured by:
    a. Fréchet Distance
    b. Routes Segment Overlap

Hypothesis

(H_0): There is no significant difference between the performance of the proposed Dynamic HC2L and the DHL algorithm in terms of the following:

a. Labeling time
b. Labeling size
c. Query response time

Scope and Limitations of the Study

This study focuses on improving optimally queried paths in dynamically changing road networks by adding a threshold-based lazy update mechanism to the Hierarchical Cut 2-Hop Labeling (HC2L) algorithm. The proposed Dynamic HC2L aims to limit unnecessary label recomputation during disruptions such as congestion, accidents, planned closures, and user incidents without compromising routing accuracy and computational speed.

The study addresses urban road networks as directed graphs, with intersections as nodes and roads as edges with computed travel time weights. The primary study site is Quezon City, Metro Manila, a large, congested city with frequent traffic disruptions. The congestion coordination issues of Quezon City, such as unstable closure management and persistent jamming (Mangahas & Medes, 2019), require adaptive routing capable of maintaining efficient routes even in frequent disruptions. Based on this, HC2L is extended with dynamic optimization techniques to enhance responsiveness under realistic scenarios. The system is tested with simulated edge-weight updates mimicking frequent disruptions, closing the gap between HC2L's theoretical potential and scalable adaptive routing demands.

The proposed Dynamic HC2L algorithm will be implemented in C++ to ensure computational efficiency and low-level memory control during label generation and updates. The simulation environment for applying disruptions, managing thresholds, and visualizing results will be developed using Python and JavaScript. The physical road layout is retrieved from OpenStreetMap, and traffic metadata (speed, jam factor, traversability) are retrieved from the HERE Traffic API. Disruptions are pre-computed and are divided into Light, Medium, and Heavy disruption levels based on rule-based thresholds. This allows for controlled testing without needing live traffic feeds.

The model accommodates only edge weight updates (i.e., pre-simulated disturbances and user-provided events mimicking real-time alerts). They are checked by a threshold mechanism to identify whether labels are updated in real-time or are delayed. On query submission, both preloaded and user-provided disturbances are taken into account in path calculation. Structural updates, i.e., addition or removal of roads, are not provided. Testing is performed in a simulation environment with pre-acquired datasets.

The Lazy Update mechanism intentionally delays label recomputation when disruptions are minor to favor efficiency, which may result in temporarily stale labels updated on demand during queries. This trade-off is deliberate and central to the experimental evaluation.

While the approach is designed to be scalable, this study compares Dynamic HC2L only to Dual-Hierarchy Labelling (DHL) within a single urban area (Quezon City). The results are therefore not intended to be generalized to country-wide, real-time applications without additional validation.

Despite these limitations, the study introduces a lightweight, adaptive pathfinding approach that balances efficiency and accuracy in traffic-prone urban networks. Its results can inform routing systems, urban traffic simulation, and research on scalable, disruptionaware navigation.

Significance of the Study

The significance of this study lies in improving the efficiency and responsiveness of real-time route planning in disruption-prone urban road networks. In fast-changing urban environments like Quezon City, traffic congestion, accidents, and road closures can quickly render static routing algorithms such as HC2L impractical due to their reliance on full-label recomputation lead to high labeling time, memory usage, and query delays. This leads to

performance delays in time-sensitive contexts, including emergency response and logistics operations.

Although dynamic labeling approaches like DHL have been developed to mitigate these limitations, they often involve substantial resource requirements and complex configurations that can be difficult to scale. To date, no lightweight and scalable solution has been widely proven optimal for frequent, localized disruptions. The proposed Dynamic HC2L, which integrates lazy updates and threshold-based triggering, offers a practical strategy to reduce computation while maintaining query accuracy. However, important questions remain regarding how delayed updates impact route correctness and under what conditions updates should be triggered. Addressing these questions is critical to advancing dynamic pathfinding methods and supporting more responsive, disruptionaware navigation in real-world urban networks.

Urban Commuters and Drivers. They will benefit from faster, more predictable routes that adapt to changing driving conditions, road closures, or hazards, thereby reducing delays and stress. Improved routing can also lead to fuel savings and reduced vehicle emissions.

Emergency Responders and Public Safety Agencies. In time-sensitive operations, real-time routing can save lives by ensuring the fastest possible access to emergency sites.

Transportation Planners and Local Authorities. Providing accurate information on the shortest path greatly supports traffic management, urban planning, and public transport system decision-making.

Logistics and Delivery Services. Dynamic routing can improve on-time deliveries and reduce operational costs, driving competitiveness and enhancing customer satisfaction in an increasingly demanding marketplace.

The proposed system enhances dynamic routing by reducing query response times through targeted updates, minimizing update delays by prioritizing major incidents, and

optimizing memory use by storing only incremental changes. These improvements support a scalable, real-time solution suitable for urban environments. Utilizing real-world datasets ensures the findings are applicable to urban mobility, emergency services, and infrastructure management.

Definition of Terms

2-Hop Labeling. A method of indexing graph nodes where each node stores intermediate "hub" nodes to quickly compute the shortest path between any two nodes (Ahn & Im, 2020)

Adaptive Label Management. A proposed strategy for selectively updating only the affected parts of the label index after network changes, improving efficiency.

Balanced Partitioning. A method for dividing a graph into subgraphs of roughly equal size to optimize computational performance.

Disruption Severity Levels. A classification system that categorizes traffic incidents by their severity and impact on travel time (low, medium, high), derived from speed, jam factor, and traversability (HERE Technologies, n.d.).

Dual-Hierarchy Labeling (DHL). The baseline comparison for this study, a graph labeling method that uses separate query and update hierarchies to efficiently handle distance queries and updates in dynamic road networks (Farhan et al., 2025).

Dynamic HC2L. The proposed version of HC2L incorporates lazy updates and threshold-based mechanisms to efficiently handle dynamic changes in road networks.

Dynamic Road Network. A road network whose structure or traffic conditions change over time due to events such as congestion, accidents, or construction.

Fréchet Distance. A mathematical measure used to quantify the similarity between two curves by considering the location and order of points along their paths. In this study, it is used to compare the geometric similarity between the route (Conradi,
2024).

Hierarchical Cut 2-Hop Labeling (HC2L). A routing algorithm that uses hierarchical partitioning and 2-hop labeling to precompute distances, enabling fast shortest-path queries in large-scale static networks (Farhan et al., 2023).

Label Construction Time. The time required (in seconds) to modify or rebuild label data structures after changes in the network (Farhan et al., 2023).

Labelling Size. The total memory footprint (in megabytes) required to store all precomputed distance labels for a road network. (Farhan et al., 2023)

Lazy Updates. A mechanism where label or index updates are delayed and only recomputed when triggered by a query or significant graph change (Aioanei, 2012).

Query Response Time. The duration it takes (in microseconds) for the system to return the result of a shortest path query (Farhan et al., 2023).

Optimized Path Distance. The minimum cumulative weight (in meters) of a path between two vertices in a road network graph

Optimized Path Query. A request to compute the minimum-cost route between two points in a graph based on travel time adjustments under road incidents.

Simulated Incident Type. The predefined categories of traffic disruptions (congestion, accidents, and road closure) injected into simulation for testing.

Static Routing Algorithm. A pathfinding algorithm designed for edges that do not change over time.

Tail-Pruning Strategy. A technique used in labeling to reduce memory usage by removing redundant or infrequently used path information (Farhan et al., 2023).

Threshold-Based Triggering. A rule-based system where updates occur only when network changes surpass a predefined impact threshold (Stan et al., 2023).

Threshold Value (τ). A predefined value that determines when label updates are triggered based on significant changes in travel time or disruption severity.

Type of Disruption. A predefined category of road incidents used in this study to simulate real-world traffic conditions.

Vertex-Cut Graph. A graph partitioning approach where certain key vertices are removed to divide the graph into smaller, more manageable subgraphs.

Chapter 2

REVIEW OF LITERATURE AND STUDIES

This chapter provides a comprehensive review of existing literature and studies related to shortest path computation, particularly within the context of graph theory and transportation networks. As modern systems increasingly demand efficient, real-time route planning, especially in logistics, navigation, and emergency response, the need for optimized algorithms has grown significantly. This chapter explores the evolution of shortest path algorithms from classical methods like Dijkstra's and A* to more advanced hierarchical and labeling-based techniques, including recent innovations tailored for dynamic and time-dependent road networks. By examining the strengths and limitations of these approaches, this review establishes the foundational knowledge necessary for understanding the current challenges and research directions in efficient pathfinding solutions.

Fundamental Approaches to Shortest Path Computation

Classical Search-Based Algorithms.

Existing approaches to solving the shortest path problem, particularly in transportation networks and graph theory, have been extensively studied. One of the most used methods to solve this problem was using Dijkstra's algorithm. It is designed to find the shortest path from a single source node to all other nodes in a graph with non-negative edge weights (Alam & Faruq, 2019). The algorithm operates by iteratively selecting the node with the smallest tentative distance, updating the distances of its neighboring nodes, and marking it as visited, ensuring that the shortest path is systematically determined

(Khan, 2020). This method is particularly effective for static graphs and has been applied in various domains, including road networks, computer networks, and logistics optimization (Dobrilovic et al., 2012).

However, the classical Dijkstra's algorithm has limitations, such as its inability to handle graphs with negative edge weights and its computational inefficiency for largescale networks due to its time complexity of $O(V^2)$ when implemented with simple arrays (Rachmawati & Gustin, 2020). To address these limitations, researchers have developed optimized variants and alternative algorithms. Among these, the A* algorithm stands out by incorporating heuristic estimates (e.g., Euclidean distance) to guide the search process. Unlike Dijkstra's exhaustive approach, A* prioritizes nodes likely to lead to the goal, significantly reducing the search space. Empirical studies demonstrate its superiority in large-scale maps, where it achieves faster computation while maintaining optimality, — provided the heuristic is admissible (Wayahdi et al., 2021; Rachmawati & Gustin, 2020). However, A*'s reliance on problem-specific heuristics can limit its applicability in graphs lacking spatial metadata, prompting further innovations like hierarchical or dynamic adaptations (Gbadamosi & Aremu, 2020).

Advanced Graph Algorithms for Real-World Navigation

Navigation Systems and Their Underlying Algorithms.

Modern navigation systems like Google Maps and Waze exemplify the practical application of shortest path algorithms. These widely used mobile applications provide drivers with real-time route alternatives to determine the fastest or easiest path (Trapsilawati et. al., 2019). Waze allows users to report accidents, traffic jams, road works, and hazards, with these alerts subject to user evaluation (Laor & Galily, 2022). However, raw Waze data often contains repeated reports and may lack high reliability, necessitating data preprocessing (Amin-Naseri et. al., 2018). Furthermore, while Waze can tune components of the A* algorithm for performance improvements, such as pathfinding speed

and memory use, research suggests merging RPT and JPS algorithms with A* to filter superfluous nodes, potentially reducing execution time by 67% and improving pathfinding performance by 47% (Wei et. al., 2024).

Google Maps, on the other hand, relies on extensive ground data to find optimal routes, meticulously documenting roads and highways and gathering real-time data through continuous patrolling (Trapsilawati et. al., 2019). While Google Maps excels in providing highly accurate routes, it cannot include a user's personal speed and can only estimate arrival times based on factors like distance remaining, average speed due to real-time traffic, historic data, and officially recommended speeds (Mehta et. al., 2019). Notably, Google Maps also employs the A* method for shortest path computation due to its high accuracy, flexibility, and capacity to handle large graphs, outperforming Dijkstra's approach. Both Google Maps and Waze provide positioning information and involve significant human-computer interaction, where Human-Computer Trust (HCT) is crucial for effective system use (Trapsilawati et. al., 2019).

Hierarchical Indexing Techniques.

To overcome the limitations of direct search methods like Dijkstra's algorithm, particularly in large-scale road networks, hierarchy-based solutions have been developed. Contraction Hierarchy (CH) is a state-of-the-art indexing-based approach that exploits the inherent hierarchical structure of road networks. It involves a preprocessing step where shortcut edges are added to the network (Geisberger et. al, 2012).

Building upon this concept, Hierarchical Labeling (HL) assigns labels to vertices while maintaining a hierarchical structure among them. This approach allows for efficient query processing by selectively visiting parts of the labels based on the vertex hierarchy (Abraham et al., 2012). Theoretical advancements in hierarchical hub labeling have led to faster preprocessing algorithms, making this approach more practical for a wider range of graphs.

Advanced Labeling Techniques for Road Networks

Hub-Based Labeling Methods.

Hub Labeling (HL) assigns every node a list of hubs with precomputed distances, allowing shortest path queries to be responded to efficiently by leveraging common hubs. This index method considerably accelerates query time, frequently outperforming conventional approaches by an order of magnitude (Li et al., 2017). However, HL is primarily designed for static networks, and its dependence on edge weights in label computation makes it less suitable for dynamic networks where updates have to be made often (Babenko et al., 2015).

Pruned Highway Labeling (PHL) is an extension of the hierarchical labeling approach (Akiba et al, 2014) that aims to further reduce label sizes and improve query performance in road networks. The Hierarchical 2-Hop Index (H2H) is another novel approach that combines elements of both hierarchy-based and hop-based solutions. H2H assigns labels to each vertex while preserving a hierarchy among all vertices, effectively addressing the drawbacks of previous methods (Ouyang et al., 2018).

Optimized Labeling Structures.

Highway Cover Labeling (HCL) addresses some issues by targeting highway nodes to cover long-distance paths, leading to smaller labels and quicker queries in static networks. Farhan and Wang (2023) presented a dynamic variant supporting quicker updates using parallel search and focused repairs. However, HCL still relies on fixed hierarchies, which make it less flexible in irregular or dynamic networks.

Tree Decomposition Labeling (H2H), specifically the Hierarchical 2-Hop Index (H2H), improves query performance by giving 2-hop labels while maintaining a hierarchy between vertices so that queries can access only an appropriate subset of labels (Ouyang et al., 2018). The same paper underscores the limitations of hierarchical methods in

dynamic or non-uniform settings, observing structural brittleness in other graph classes where wide cores or ill-structured graphs make decomposition challenging.

P2H builds upon Chen et al.'s work (2021) by extending this concept to project vertex separators, further minimizing label size and incorporating update-efficient indexing. Partitioned Dynamic Hub Labeling (ParDHL) (Zhang et al., 2025) and CPU-GPU hybrid methods (Li et al., 2024) provide flexibility for dynamic environments, although they come at the expense of either higher maintenance complexity or dependence on specialized hardware.

Hierarchical Cut 2-Hop Labeling (HC2L).

Hierarchical Cut 2-Hop Labeling (HC2L) is a labeling-based approach designed to accelerate shortest-path distance queries on road networks (Farhan et al., 2023). It introduces a balanced tree hierarchy constructed through recursive partitioning of the graph into balanced subgraphs using minimal vertex cuts. This hierarchy ensures that each partition maintains accurate distances via shortcuts between border vertices, forming a binary tree where internal nodes represent cuts. Queries leverage the lowest common ancestor (LCA) of two vertices in this tree, which inherently contains a hub vertex on their shortest path. By restricting label searches to hubs associated with the LCA, HC2L drastically reduces query search spaces.

Additionally, tail pruning eliminates redundant distance entries in labels, prioritizing critical hubs and shrinking label sizes by up to 60% compared to state-of-the-art methods like H2H and PHL. These optimizations enable HC2L to achieve query speeds 1.5–4× faster than competitors. Hierarchical Cut 2-Hop Labeling (HC2L) integrates 2-hop indexing and hierarchical graph cuts to form succinct labels, facilitating quick query times, low memory usage, and improved scalability to large graphs (Farhan et al., 2023). In contrast

to its ancestors, HC2L provides improved support for dynamic updates without the inflexible reliance on fixed hierarchies.

However, HC2L assumes static edge weights, making it unsuitable for dynamic road networks where real-time factors like traffic or accidents alter edge costs. Its reliance on precomputed labels and a fixed hierarchy limits adaptability to such changes, necessitating full recomputation for updates, a computationally prohibitive process. This gap underscores the need for extensions to support dynamic edge weights.

Addressing Dynamic Environments: Challenges and Adaptive Strategies

Computational Challenges in Dynamic Environments.

Shortest path computation in time-dependent road networks (TDRNs) is a core problem in today's transportation networks, whose edge weights (e.g., travel time) change concerning departure time. Although static road networks are efficiently processed by Dijkstra's or A* algorithms with $O(m + n \log n)$ time complexity, TDRNs need real-time responsiveness to dynamic traffic changes, rendering precomputed indexes useless and traditional methods impractical in networks with over one million nodes (Wang et al., 2019).

The primary problem is to find a balance among speed, accuracy, and scalability while addressing three primary challenges: (1) temporal dependency, whose traffic variations can render cached paths stale in minutes; (2) memory constraint, where hierarchical indexing schemes (e.g., TD-G-tree) suffer from excessive storage and long preprocessing times (Gong et al., 2023); and (3) the requirement of real-time responsiveness, whose dynamic adaptation in methods like Hub Labeling (HL) fails to prioritize key routes, leading to wasteful usage of resources in redundant recalculation (Zhang et al., 2021).

Table 1

Comparative Table for Short Path Queries

Method	Preprocessing Time	Scalability	Real-Time Updates	Accuracy	Hardware Dependency	Key Limitations
Dijkstra/A*	None	Low	No	Optimal	CPU	Inefficient for large networks
TD-G-tree	High	Moderate	Partial (slow recompute)	Optimal	CPU	Prohibitive storage and preprocessing costs
Hub Labeling	Very High	Low	Yes (no prioritization)	Optimal	CPU	Redundant recalculations; fails to prioritize critical routes
G2H	Low (GPUparallel)	High	Yes (fast index rebuilds)	Suboptimal (≤5%)	GPU	Accuracyhardware trade-offs

Performance Requirements in Real-World Systems.

The significance of shortest path algorithms in real-time systems highlights the essential demand for both speed and precision. In the logistics sector, suboptimal routing leads to increased operational expenses due to fuel inefficiency and delayed deliveries, a problem exacerbated in congested networks where conventional algorithms struggle to perform effectively (Li et al., 2024). Inefficient routing for ride-hailing harms driver revenue and consumer happiness, especially in congested areas where quick route changes are essential (Li et al., 2024).

Additionally, emergency response systems need to be updated with millisecond precision. Katre & Thakare (2017) point out that even small delays in emergency vehicle routing might endanger lives because, in emergencies, the exact quickest path, rather than merely rough estimates, is needed. Their suggested "Quick Response Device" demonstrates how inefficient or outdated algorithms can compound delays in

emergencies, where every second counts. These applications not only require speed but also context-aware accuracy. Consumer navigation apps can be suboptimal by a little (e.g., 5% longer distance), but emergency systems demand accuracy, as underscored by Katre & Thakare (2017). GPU-based architectures such as G2H (Li et al., 2024) cover this shortfall by providing fast index rebuilds and query answers and guaranteeing real-time responsiveness to changing road conditions.

Dynamic Variants of Labeling-Based Algorithms

Dual-Hierarchy Labeling (DHL).

Dual-Hierarchy Labelling (DHL) has emerged as a significant advancement for handling distance queries on dynamic road networks, addressing critical limitations in existing methodologies, particularly those related to maintaining efficiency in large-scale networks under frequent updates (Farhan et al., 2025). DHL operates by establishing two distinct yet complementary hierarchies: a query hierarchy (HQ) and an update hierarchy (HU), effectively separating query processing from update maintenance. This dualhierarchy approach is fundamentally different from earlier solutions, such as Dynamic Contraction Hierarchies (DCH) (Wu et al., 2015) and Incremental Hierarchical 2-Hop (IncH2H) (Zhang et al., 2020), which often struggle to balance query efficiency and maintenance overhead (Farhan et al., 2025).

The query hierarchy (HQ) within DHL is structured as a balanced binary tree created through recursive partitioning, exploiting minimal balanced cuts. This hierarchical structure significantly reduces query processing times by limiting search spaces to labels of common ancestors for query vertices, a strategy derived from hierarchical 2-hop indexing principles (Ouyang et al., 2018). Conversely, the update hierarchy (HU) is maintained dynamically as a shortcut graph using vertex partial ordering based on the query hierarchy, allowing DHL to quickly propagate and localize the impact of edge

updates, whether weights increase or decrease (Farhan et al., 2025). This partial ordering substantially simplifies update processes compared to methods relying on total vertex ordering, like IncH2H, which often face challenges with scalability and complex maintenance mechanisms (Zhang & Yu, 2022).

A notable feature of DHL is its hierarchical labelling (L), which employs a sophisticated compression strategy to store distances within induced subgraphs rather than the entire graph. By restricting distances to subgraphs, DHL ensures that updates affect only local regions of the hierarchy, markedly reducing the number of labels requiring updates during edge changes (Farhan et al., 2025). This approach contrasts sharply with IncH2H and DCH, which store comprehensive distances across larger graph segments, resulting in significantly larger labels and slower update performance.

Empirical evaluations on extensive road networks from the USA and Europe indicate that DHL outperforms state-of-the-art methods by achieving 2-4 times faster query processing and 3-4 times faster updates, using only 10%-20% of the labelling space (Farhan et al., 2025). Such performance gains highlight DHL’s strengths in terms of reduced update overhead, faster query response, and compact memory usage. However, DHL does present certain limitations. Specifically, DHL's hierarchical labels necessitate careful balancing to ensure optimal efficiency, particularly in the presence of frequent structural changes like edge insertions or deletions. Although edge deletions and insertions are relatively infrequent in practical road networks, managing these efficiently remains a challenge that DHL partially addresses by adjusting hierarchy structures through localized repartitioning and shortcut adjustments (Farhan et al., 2025).

Despite these advantages, some gaps persist in the DHL framework that justify further research into new dynamic pathfinding methods. Notably, DHL’s method of recomputing label entries when the shortest paths are not unique may lead to unnecessary computational overhead in rare but significant cases. Further research is needed to

explore how to efficiently track and leverage support structures without incurring significant additional memory or computational overhead, a challenge already partially addressed by other methods such as IncH2H (Zhang & Yu, 2022).

Given these strengths and limitations, DHL serves as an appropriate baseline for evaluating newly proposed algorithms. Its balance of efficient query processing, minimal memory overhead, and relatively low update cost positions it as a comprehensive benchmark. Future algorithms must therefore demonstrate clear improvements in label compression efficiency, update speed, and reduced computational overhead in managing dynamic changes to surpass the established performance of DHL.

Stable Tree Labelling (STL).

Recent research by Koehler et al.(2025) has introduced the Stable Tree Hierarchy and Stable Tree Labeling (STL) as a novel approach for efficient shortest path querying in dynamic networks. STL presents a significant advancement by decoupling the hierarchical structure from edge weights, resulting in a structurally stable tree even amidst dynamic network changes. This separation allows updates to be localized to intra-subgraph distances, avoiding global recomputations and thus substantially improving computational efficiency.

One of the key advantages of STL lies in its reduction of update complexity. Traditional methods often require reprocessing the entire network or large portions of it when changes occur. In contrast, STL confines updates to localized labels within subgraphs. This not only minimizes recomputation costs but also enhances responsiveness in real-time applications. Moreover, space efficiency is achieved by storing only intra-subgraph distances instead of full-path metrics, making the method scalable to large networks. Koehler et al. also introduce specialized algorithms, namely Label Search and Pareto Search, which support fast query and update operations with minimal graph

traversal. These algorithms demonstrate significant gains in empirical performance, with STL-based methods outperforming previous approaches in both query and update times across ten large-scale real-world road networks.

Despite these strengths, STL has its limitations. The scope of dynamic changes considered is primarily restricted to weight updates. While the authors suggest that STL may be adaptable to topological changes such as edge or vertex insertions and deletions, concrete algorithms and theoretical guarantees for such operations remain underdeveloped. Another limitation concerns network applicability. The method is validated mainly on road networks, which tend to exhibit stable and sparse connectivity. However, its performance and adaptability to highly dynamic networks, such as social or communication graphs with frequent topology shifts, are yet to be evaluated.

Furthermore, the algorithmic optimality and theoretical bounds of STL remain loosely defined. While empirical results suggest superior performance, formal analyses of worst-case complexities, label sizes, and query guarantees are not rigorously provided. This leaves uncertainties regarding STL’s performance under extreme or adversarial conditions.

Lastly, STL's current scope is limited to undirected, weighted graphs. Its extension to directed or more structurally complex graphs, particularly those with non-uniform dynamics or fluctuating connectivity, presents an open area for future work. In such networks, the stability assumptions of the hierarchy may no longer hold, which could negatively affect STL’s efficiency.

Full and Partial Recomputation: Time and Resource Implications

As urban road networks become increasingly dynamic due to congestion, accidents, and infrastructure developments, efficiently updating the underlying network models has become critical. This subsection compares full and partial recomputation

strategies in terms of time complexity and resource consumption, especially in the context of real-time navigation and traffic-aware applications.

Full Recomputation of Road Networks.

Full recomputation involves recalculating the entire graph structure and all associated paths from the ground up, regardless of the nature or location of the changes. This approach offers algorithmic simplicity and guaranteed global optimality, making it particularly suitable for handling large-scale disruptions such as natural disasters or infrastructure failure. Algorithms like Dijkstra's or Floyd-Warshall ensure that the recalculated paths are optimal within the current network state (Fan et al., 2020).

However, this strategy is often computationally expensive, particularly for large datasets such as the Road-CA graph, which contains approximately 1.97 million vertices and 2.77 million edges. The time complexity of Dijkstra's algorithm, expressed as $O(V \log V + E)$, or Floyd-Warshall's $O(V^3)$, makes full recomputation impractical for systems that require near-instantaneous updates (Wang et al., 2023). This limitation is especially critical in real-time applications where responsiveness is essential, such as emergency routing or dynamic traffic rerouting.

Partial Recomputation for Dynamic Networks.

In contrast, partial recomputation, also referred to as incremental computation, addresses efficiency concerns by updating only the affected segments of the network based on localized changes. This approach is based on the idea that small changes in the graph (ΔG) usually result in proportionally small output differences (ΔO), which reduces the need to recompute the entire dataset. Such optimization is particularly advantageous in environments characterized by frequent but minor disruptions, such as temporary traffic congestion or localized road closures.

Incremental graph algorithms are specifically designed to limit the scope of computation by focusing only on affected nodes and edges. This results in significant time savings and increased responsiveness (Zhou et al., 2021). However, partial recomputation introduces greater algorithmic complexity because it requires careful management of data dependencies, change propagation, and consistency enforcement. These additional challenges can increase development time and maintenance effort, particularly in large-scale systems.

Adaptive Update Strategies for Dynamic Labeling

Limitations of Traditional Approaches.

Despite the effectiveness of methods such as HL, PHL, and H2H for static networks, they require costly updates when edge weights change, making them unsuitable for dynamic environments (Ouyang et al., 2018). Although DHL improves update performance through separate query and update hierarchies (Farhan et al., 2025), few approaches can handle the real-time demands of traffic routing, disaster response, or smart city applications (Huang et al., 2021). However, despite advances in labeling approaches, dynamic distance querying remains an underdeveloped area, as most methods still rely on expensive recomputation, further limiting their applicability in realtime, dynamic networks.

Selective Update Mechanisms.

The efficient management of dynamic graphs necessitates strategies to minimize computational overhead by updating information selectively and only when necessary. This principle aligns with the concept of "lazy updates," which delay or avoid recomputation until strictly required. Earlier works such as Aioanei (2012) explored lazy shortest path computation in dynamic graphs, focusing on updating shortest paths incrementally as edge weights change, thereby reducing redundant computation.

More recent advances have extended these ideas substantially. Dynamic graph algorithms now leverage incremental and decremental update mechanisms that localize recomputation to affected parts of the graph, avoiding full recalculations. For example, van den Brand et al. (2023) introduced dynamic algorithms with predictions, which use imperfect forecasts of future updates to optimize update and query times for problems such as incremental transitive closure and approximate all-pairs shortest paths (APSP). Their approach achieves near-constant worst-case update time and query times dependent on prediction error, representing a significant step forward in selective update efficiency.

Furthermore, fully dynamic graph algorithms developed recently maintain exact shortest paths and other graph properties against adaptive adversaries with subquadratic update times by reconstructing paths from a small number of distance estimates (Henzinger et al., 2023; Li et al., 2024). These methods improve upon classical approaches by combining lazy update principles with advanced data structures and prediction models. In addition, dynamic graph databases and knowledge graph construction methods have incorporated selective update mechanisms to handle out-oforder updates and evolving data efficiently, as seen in recent works on in-memory dynamic graph databases (Zhang et al., 2024) and dynamically updatable knowledge graphs (Kumar et al., 2025). Lastly, traffic thresholding mechanisms for congestion avoidance, such as those studied by Stan et al. (2023), also embody selective update principles by triggering updates only when congestion exceeds certain thresholds, thus reducing unnecessary computations in dynamic hierarchical labeling systems.

Threshold-Based Optimization Techniques.

Thresholds are a proven mechanism for managing updates in continuous monitoring systems. The traffic thresholding mechanism proposed by Stan et al. (2023) demonstrates how thresholds can reduce congestion, system overhead, and CO_2

emissions by delaying updates until significant changes occur. A similar concept is discussed in Tang et al. (2012), who applied dual-threshold monitoring in distributed probabilistic systems to minimize communication costs by only reacting when critical values exceed predefined limits. Building on this foundation, recent research has further refined thresholding strategies. Zhang et al. (2022) enables automatic, communicationefficient distributed monitoring of arbitrary functions by minimizing update messages while preserving accuracy, demonstrating the evolution of selective update strategies in distributed environments. These advances continue to embody the principle of selective updates by updating only when necessary, thus optimizing system performance in dynamic environments. While their domain differs, the underlylIng principle of selective, threshold-driven updates supports our motivation to apply this mechanism to hierarchical 2-cut labeling, where updates are triggered only when edge weight changes surpass a defined magnitude, thus optimizing computational efficiency.

Synthesis of the Reviewed Literature and Studies

The development of shortest path algorithms has progressed from foundational techniques such as Dijkstra's and A* to more advanced hierarchical and labeling-based frameworks. Classical algorithms remain effective for static graphs, but their high computational cost limits scalability in large or frequently changing networks (Alam and Faruq, 2019; Rachmawati and Gustin, 2020). To address these limitations, hierarchical indexing approaches such as Contraction Hierarchies (Geisberger et al., 2012) and Hub Labeling (Li et al., 2017) use precomputation to accelerate query times. However, these methods are mainly designed for static environments. Even small updates in edge weights often require full label recomputation, which results in significant processing overhead (Babenko et al., 2015). Recent methods like Hierarchical Cut 2-Hop Labeling or HC2L (Farhan et al., 2023) achieve high performance on static networks but still carry the same

limitations, making them less suitable for applications with frequent changes in travel conditions.

Real-time systems such as traffic navigation and emergency routing demand fast and adaptive responses. These requirements expose critical limitations in existing algorithms. Although some methods such as Partitioned Dynamic Hub Labeling (Zhang et al., 2025) and GPU-accelerated label construction (Li et al., 2024) offer dynamic capabilities, they often introduce increased complexity or depend on specialized hardware. Stable Tree Labeling techniques and Dual-Hierarchy Labeling (Farhan et al., 2025) also attempt to optimize responsiveness through localized updates or separated query-update hierarchies, but these approaches tend to involve complex index management or may not scale well under rapid, high-volume change scenarios. These factors limit their scalability and practical use (Gong et al., 2023; Wang et al., 2019). In systems where timing is crucial, even slight delays can lead to serious consequences (Katre and Thakare, 2017). This situation shows the need for a more flexible and efficient solution.

Table 2 Summary of the comparative characteristics of prominent static and dynamic shortest-path algorithms

Algorithm	**Update Strategy**	**Scalability**	**Query Speed**	**Real-Time Readiness**	**Hardware Dependency**
Dijkstra/A*	None	Low	Slow	No	Low
HL	Full Recompute	Moderate	Fast	No	Moderate
PHL/H2H	Full Recompute	Moderate	Fast	No	Moderate
ParDHL	Partitioned Update	High	Fast	Partial	Moderate

STL	Localized Intrasubgraph Update	High	Fast	Yes	Low
DHL	Dual Hierarchy	High	Very Fast	Limited	Moderate
G2H	GPU accelerated	High	Very Fast	Yes	High
HC2L	Static Hierarchy	High	Very Fast	No	Low

A key research gap exists in current approaches. Most systems either focus on fast query performance in static settings or on dynamic adaptability at the cost of high computational load, reduced accuracy, or added hardware requirements. No existing framework fully balances high-speed query execution with efficient updates suitable for continuously changing road networks.

This study proposes an enhanced version of the HC2L framework that applies lazy updates and threshold-based triggers. This approach is based on the principle of selective computation, where updates are performed only when changes reach a meaningful threshold (Aioanei, 2012; Hsueh et al., 2010; Stan et al., 2023; Tang et al., 2012). By combining HC2L's fast query structure with lightweight and adaptive update mechanisms, this method aims to improve both performance and responsiveness in dynamic road network applications while keeping computational and communication costs low. The selection of HC2L as the base algorithm is also grounded in its structural suitability for urban environments such as Metro Manila. Its binary tree-based graph partitioning and use of vertex cuts mirror the hierarchical nature of real-world road systems, where local streets, arterial roads, and highways follow layered traffic flows. This makes HC2L not only efficient for static querying but also structurally compatible with adaptive extensions that require localized updates.

Chapter 3

METHODOLOGY

Research Design

This study employs a quantitative comparative experimental research design for designing and testing a dynamic expansion of the Hierarchical Cut 2-Hop Labeling (HC2L) algorithm. The focus is on testing the efficiency, responsiveness, and routing accuracy of the algorithm on road networks in simulated disruption-prone road conditions. The road network is simulated as a directed graph $G=(V,E)$, with road segments as edges and travel time as weights. The weights are dynamically updated with traffic metadata from the HERE Traffic Flow API, with disruptions simulated by a rule-based process considering speed drops, jam factors, and traversability status. Disruption scenarios vary both in type and severity, with severity levels Light, Medium, or Heavy. Composite disruptions (e.g., accident and congestion) are applied when two or more conditions are present on a single segment to better simulate complicated urban traffic behavior.

Independent variables of the experiment are severity level of disruption (Light, Medium, Heavy), type of disruption (e.g., congestion, accident, road closure), employed algorithm (Dynamic HC2L vs. Dual-Hierarchy Labeling (DHL)), and employed threshold value (τ) for label update. Dependent variables for SOP 1 and SOP 2 for performance comparison are labeling time in seconds, labeling size in megabytes, and query response time in microseconds. For SOP 3 route similarity comparison, dependent variables are Fréchet Distance between estimated route and a route in Google Maps and route segment overlap in percentage of common segments with reference route. Controlled variables are

road network topology (the Quezon City road network of OpenStreetMap), the same fixed number of source–target node pairs utilized throughout all trials, and a filtering rule so that only HERE traffic data entries with confidence of 0.7 or greater are utilized. For the simulation of actual dynamic conditions, disruption is not applied uniformly but is localized over segments to enable the existence of Light, Medium, and Heavy disruption and various types of incidents within a single trial. The algorithm is tested over several trials, each with instances of mixed disruptions varying, including user-reported incidents keyed in manually to represent real-world feedback. Both DHL (taken as the baseline) and the Dynamic HC2L method proposed here are tested under the same scenario with the same disrupted graph inputs. All the simulations are executed within a controlled environment, the core algorithm written in C++, the simulation system and the visualization modules written in Python and JavaScript. Final outcomes are recorded and statistically compared based on paired t-tests and performance visualizations to determine if Dynamic HC2L significantly outperforms DHL in terms of efficiency and routing accuracy.

Sources of Data

The data used in this study was obtained from three primary sources: OpenStreetMap (OSM), the HERE Traffic Flow API, and the Google Maps Directions API.

OpenStreetMap provided the foundational road network for Quezon City, including spatial data such as intersections (nodes), road geometries (edges), and street names. This data was extracted and preprocessed using the OSMnx Python library, which enabled the creation of a clean, directed road graph compatible with shortest-path labeling techniques (Boeing, 2017).

HERE Technologies provides real-time traffic flow data for the Philippines, which includes crucial metadata for simulating dynamic traffic conditions. Although HERE supports a wide array of global incident types, such as accident, construction, congestion, and roadClosure, incident data is not available in the Philippines, as confirmed by HERE’s

official documentation and product support (HERE Technologies, n.d.). As a result, this study did not rely on HERE's incident API but instead applied a rule-based simulation framework using only traffic flow parameters.

The following fields were extracted from the HERE Traffic Flow API and updated every 1–2 minutes:

1. speed – current estimated travel speed (m/s)
2. freeFlowSpeed – expected speed under uncongested conditions
3. jamFactor – numeric congestion level from 0.0 to 10.0
4. traversability – open/closed status of a segment
5. confidence – quality rating of the measurement

These values were filtered to retain only entries with confidence scores ≥ 0.7, following HERE's recommended best practices for reliable real-time traffic analysis (New Enhancements in HERE Traffic Analytics – Speed Data, 2024; How to Enrich Your Applications With HERE API, 2023). HERE's traffic data is globally recognized for its accuracy, combining over 160 international providers, fleet data, government sensors, and more (Michael Bauer International GmbH, 2025).

To simulate disruptions realistically, incident types were synthesized by mapping combinations of flow parameters to representative disruption categories based on HERE's incident structure. For example: jamFactor > 7 and speed < 5 km/h = Congestion. Disruptions were not applied uniformly or by severity level alone. Instead, the road network was populated with heterogeneous, mixed-type disruptions, with different types and severities occurring simultaneously across different road segments. This simulation strategy mirrors the unpredictable nature of real-world urban traffic, where congestion, road closures, and localized anomalies can coexist and overlap in effect.

Additionally, a User Report Interface was implemented to simulate crowd-sourced feedback, such as accident or hazard reports triggered when users encountered severe traffic conditions. These user reports were processed by the system's Network Change Detector and treated similarly to flow-based updates, allowing the routing system to respond to both system-detected and user-simulated disruptions.

To evaluate the route realism of the proposed Dynamic HC2L algorithm, this study incorporated the Google Maps Directions API. For each origin-destination pair used in the experiment, a real-world route was generated using Google's API with current traffic data. Since direct access to GPS-based trajectory logs in the Philippines was not available, the Google API served as a realistic proxy for driver behavior.

This combination of authoritative traffic flow data, rule-based synthetic incident modeling, and external routing reference allowed for a comprehensive and realistic simulation environment in which both Dynamic HC2L and DHL could be evaluated fairly and rigorously.

Research Instrument

To answer the research questions of this study, the researchers developed a simulation-based experimental framework that evaluates the performance of the proposed Dynamic HC2L algorithm in handling real-time edge weight changes in road networks. This framework includes both software-based tools and structured experiment tables to measure performance across defined disruption scenarios. These instruments were essential for collecting consistent data and validating the effectiveness of the Dynamic HC2L enhancements compared to the original Dual-Hierarchy Labeling (DHL) baseline.

Software Tool.

The implementation of the Dynamic HC2L and DHL algorithms was carried out using C++, due to its efficiency in handling large-scale graph structures and

high-performance computations. The broader simulation system, including the disruption simulation layer and the web-based user interface, was developed using Python and JavaScript. This hybrid setup allowed for seamless backend integration while providing an interactive frontend for data entry, visualization, and route evaluation.

The road network for Quezon City was extracted and processed using the OSMnx Python library, which allowed for efficient access to OpenStreetMap data. The road graph was modeled using the NetworkX library, where intersections were represented as nodes and roads as directed edges with dynamically assigned weights based on simulated traffic conditions.

Traffic metadata from the HERE Traffic Flow API, including speed, jamFactor, freeFlowSpeed, traversability, and confidence, was integrated into the graph structure to simulate disruption. To ensure the statistical validity of findings, NumPy and Pandas were utilized for efficient handling of matrices, label structures, and tabular performance metrics. A lightweight SQLite database was used to store intermediate results and experimental logs. SciPy was used to perform hypothesis testing and significance analysis. Visual comparisons between DHL and Dynamic HC2L were produced using Matplotlib and Seaborn, which enabled clear presentation of results across disruption scenarios. The entire project was managed under Git version control, with a GitHub repository to support collaboration and maintain code integrity.

Experiment Paper.

For collecting the required evaluation metrics, experimental tables were constructed with a view to assist every study objective. They were categorized into two classes: whole-graph metrics (labeling size and time, for instance) and querybased metrics (query response time and route similarity). Every trial

represented a varied configuration of disruption types and severity levels on the Quezon City road network. The experiment further involved threshold value testing (τ) applied in Dynamic HC2L to identify when label updates are to be initiated due to traffic condition changes. These threshold values were considered part of the independent variables and tested for various trials. Real filled-in versions of these tables are included in the Appendix, but a sample structure is depicted below.

Table 3

Sample Experiment Paper: Labeling Size and Construction Time

Trial Set	Disruption Type	Severity Level	Algorithm	Labeling Size (mb)	Construction Time (s)
Trial 1					

Table 4

Sample Experiment Paper: Query Response Time, Path Similarity (Fréchet Distance), and Segment Overlap

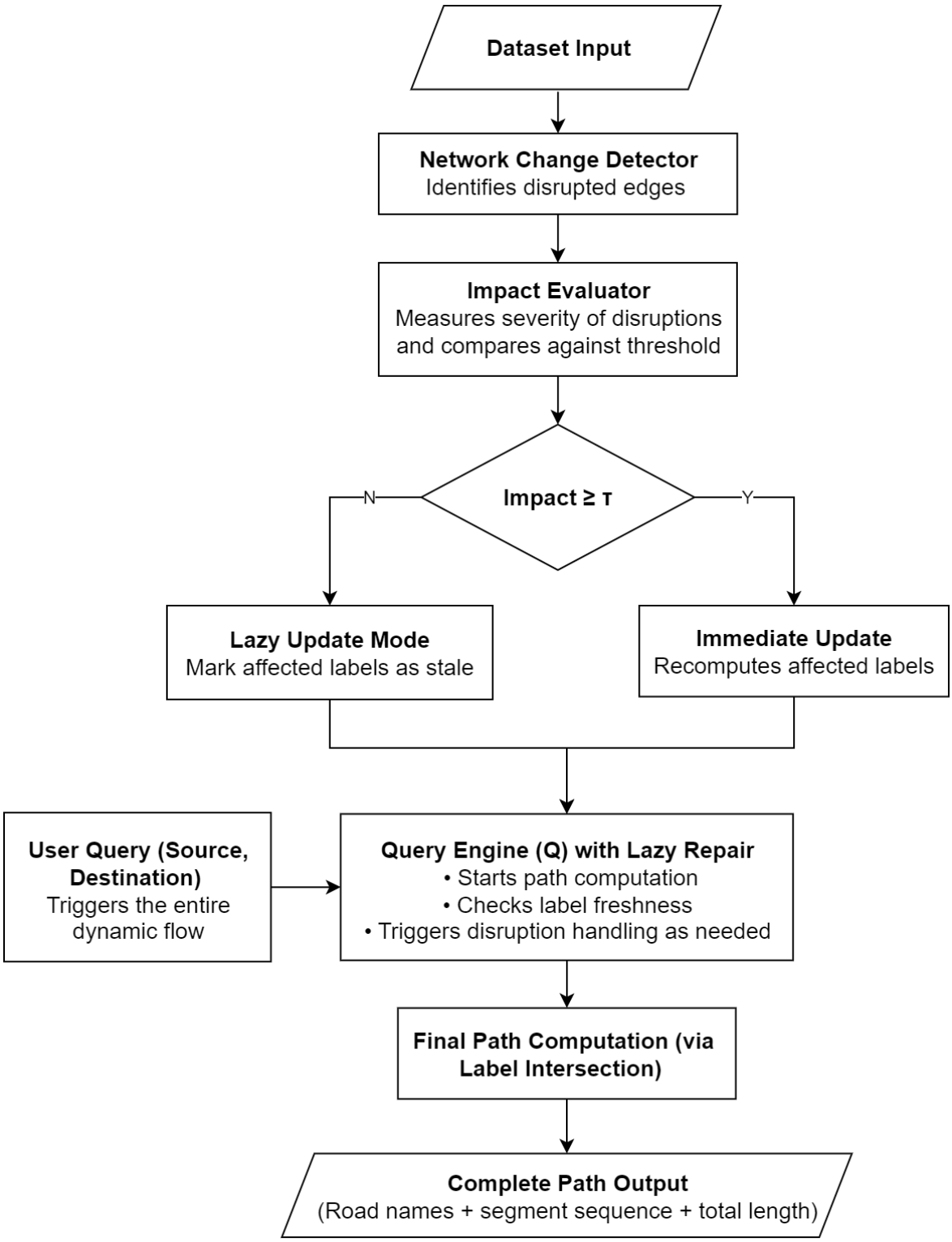
Trial	Query Pair ID	Source Node	Target Node	Disruption Type	Algorithm	Query Response Time (μs)	Fréchet Distance	Segment Overlap (%)
Trial 1								

Table 5

Sample Experiment Paper: Disruption Type and Severity vs. Fréchet Distance and Segment Overlap

Trial Set	Disruption Distribution	Threshold (τ)	Label Updates Triggered	Avg. Query Time (μs)	Avg. Labeling Time (s)	Route Accuracy (vs. Google)
Trial 1						

Figure 3. System Architecture of the Proposed Dynamic HC2L Algorithm



The Dynamic HC2L architecture builds on the static pipeline by adding mechanisms for detecting and handling simulated or real-time disruptions. This

design enables the system to update or defer label recalculations depending on the severity of the detected change.

Dataset Input. The system begins with the ingestion of real-time traffic metadata format and road network topology. Data sources include road segment structures from OpenStreetMap (OSM) and disruption data such as speed drops, jam factors, and closures from the HERE Traffic API**.** These inputs provide the foundation for identifying evolving road conditions.

Network Change Detector. Monitors road segments for changes based on updated traffic attributes such as reduced speed, increased jam factor, or road closures.

Impact Evaluator. Calculates severity using: Impact Score = $f_{\Delta w} \times fj_{am} \times f_{closure}$. Each factor is derived from HERE traffic data (speed drop ratio, jam factor, and closure flag).

$$Impact\ Score = f_{\Delta w} \times f_{jam} \times f_{closure} \quad (1)$$

Where:

Table 6

Impact Score Description

Factor	Formula	Description
$f_{\Delta w}$	$\frac{v_{new} - v_{old}}{v_{old}}$	Relative drop in speed (travel time impact)
f_{jam}	$\frac{jam\ factor}{10}$	Normalizes HERE's jam factor (0–10) to a 0–1 scale

Continuation of Table 6

$f_{closure}$	1.0 if road closed, 0.5 if open	Reflects operational impact of the road: total block or partial

Threshold-Based Decision. The system uses a threshold-based mechanism to determine whether a disruption in edge weight warrants an immediate or deferred label update. After computing the **Impact Score**, the value is compared against a selected threshold τ. The result of this comparison dictates the update strategy:

- If the **Impact Score ≥ τ** → the system applies **Immediate Update Mode**
- If the **Impact Score < τ** → the system applies **Lazy Update Mode**

This mechanism helps balance responsiveness with computational efficiency, ensuring that only significant disruptions lead to immediate recomputation.

Rather than predefining a fixed threshold, this study adopts a data-driven approach. The threshold τ will be empirically determined through simulation, this study adopts a data-driven selection process. A set of ten candidate threshold values is used, defined as:

$$T=\{0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0\}$$

This selection process is guided by prior work (e.g., Stan et al., 2023), which demonstrated that disruption sensitivity varies across network conditions and that tuning thresholds within this range can significantly improve performance. The use of a ten-level threshold set from 0.1 to 1.0 is grounded in this prior work, which showed its effectiveness in congestion management, and is further strengthened by this study's empirical evaluation to identify the optimal value based on performance metrics. Each candidate threshold will be tested through simulation, and the final value of

τ will be selected based on its ability to optimize labeling time, label size, and query response time.

Lazy Update Mode. Labels are marked stale and only repaired when accessed again via query. Saves memory and computation.

Immediate Update Mode. Labels are immediately recalculated within the affected region. It proactively updates affected labels in the background, ensuring the Query Engine has fresh data ready when a user query is submitted.

User Query (Source, Destination). When a user submits a query, it triggers the entire dynamic flow of the system. This includes checking for stale labels and computing the most appropriate route given current traffic conditions.

Query Engine with Lazy Repair. The Query Engine is responsible for processing all user-submitted routing requests. Upon receiving a source and destination, it performs the following tasks:

1. Initiates optimized path computation between the specified nodes.
2. Verifies the freshness of label data associated with the query nodes.
3. If labels are stale (i.e., marked during Lazy Update Mode), it triggers a localized repair by recomputing only the necessary subgraph labels before proceeding with the path computation.

If the system has already performed an Immediate Update, the labels will be fresh, and the engine will skip the repair step, but it still performs the freshness check to ensure correctness.

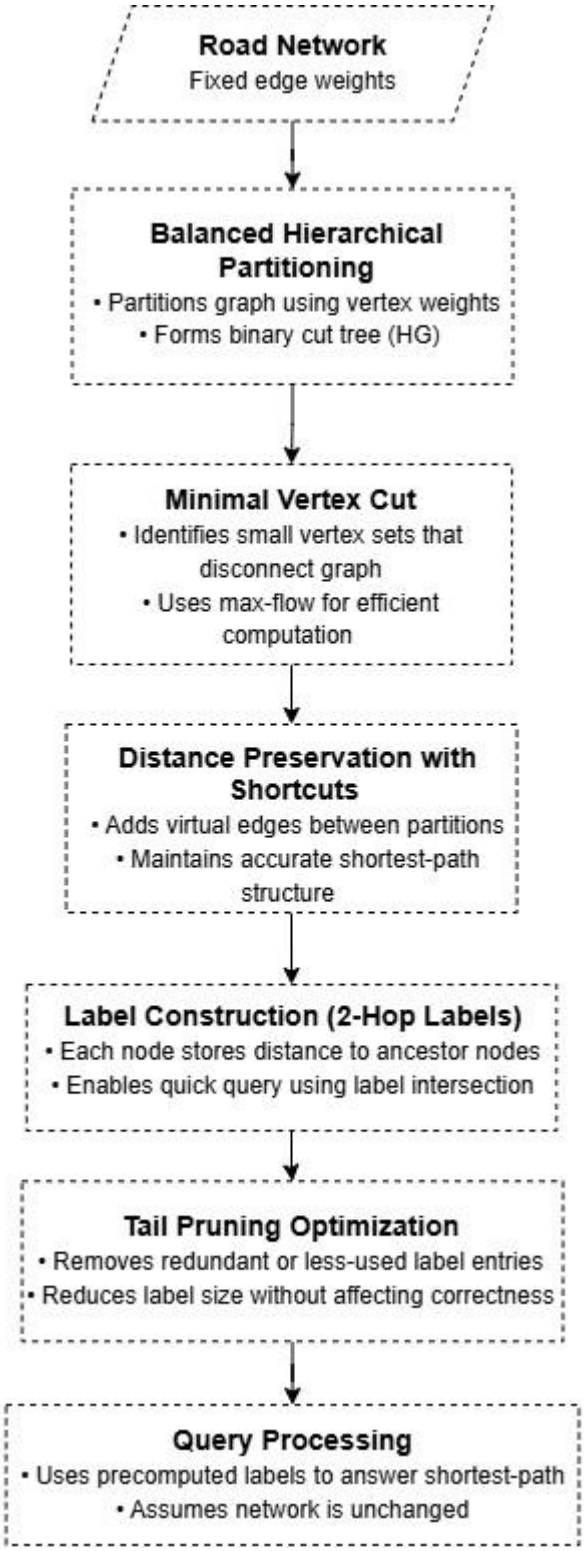
This design ensures that the Query Engine handles both preemptively updated and lazily deferred disruptions, enabling fast, accurate responses while avoiding unnecessary recomputation.

Final Path Computation (via Label Intersection). After label validation or repair, the system computes the final optimized path using 2-hop label intersection, a core principle of HC2L. This step identifies the lowest-cost route by merging forward and backward label sets through a common hub node.

Complete Path Output. Final output is human-readable road names, segment distances, and total travel time or length.

The proposed system consists of two modular architectures: the DHL framework, which supports efficient shortest-path queries on a fixed road network, and the Dynamic HC2L architecture, which integrates disruption-aware label repair using lazy and immediate updates. Both pipelines begin with a common preprocessing flow but diverge in how they handle road network changes.

Figure 4. Static HC2L Architecture



The Static HC2L model performs complete preprocessing of the road network graph before any queries are made. It assumes the graph is unchanging, which allows for efficient label-based querying but lacks adaptability to real-time disruptions.

Road Network Input. The system uses OSMnx to extract Quezon City's drivable road network, capturing node coordinates and street topology. Each road is modeled as an edge, and each intersection as a node.

Hierarchical Balanced Partitioning. The road graph is recursively divided using balanced vertex weights, forming a binary cut tree (H_G). This tree guides the hierarchical decomposition needed for label construction.

Minimal Vertex Cut Computation. Max-flow algorithms are used to compute minimal sets of separator vertices that disconnect the graph into partitions.

Distance Preservation with Shortcuts. Shortcut edges are added between partition borders to maintain shortest paths after cutting.

Label Construction (2-Hop Labels). Each node is assigned a label that stores distances to its ancestors in the hierarchy. These 2-hop labels enable fast computation via intersection at query time.

Tail Pruning Optimization. Redundant label entries are removed using a ranking mechanism based on path frequency and hierarchy depth.

Query Processing. The precomputed labels are used to answer shortestpath queries by intersecting labels and calculating combined distances.

Data Generation Procedure

Pre-Simulation Phase: Road Network and Traffic Data Preparation

The road network of Quezon City was obtained from OpenStreetMap (OSM) using the OSMnx Python library, which provided a detailed topological graph structure including road geometries, intersections (nodes), and road segments (edges). This served as the base network for all simulations. Real-time traffic data format was collected from the HERE

Traffic API, specifically from its flow data layer. Each traffic record included fields such as current speed, free-flow speed, jam factor, confidence score, and traversability status. These values were necessary to reflect congestion levels on the road network. To integrate the traffic data with the OSM network, a matching process was performed between road segments. Since road names may vary slightly across datasets, string normalization and approximate matching were used to align HERE traffic records with their corresponding OSM edges.

During Simulation Phase: Simulating Traffic Disruptions

Once the traffic data was mapped to the road network, simulated traffic conditions were applied by adjusting edge weights based on congestion severity. Although the HERE API provides flow data regularly, incident data were limited. To ensure the system was tested under realistic scenarios, synthetic closures and blockages were also introduced. Traffic congestion levels were categorized into three classes: Light, Medium, and Heavy, based on speed and jam factor values. This classification was guided by definitions from the HERE Traffic API Developer Guide (2025) and previous literature. As explained by Impedovo et al. (2019), qualitative descriptors such as “light” or “heavy” congestion are commonly associated with delays or speed reductions compared to free-flow conditions. These thresholds help determine whether a route remains usable or if rerouting is necessary. The jam factor, which ranges from 0 (free flow) to 10 (standstill), served as the primary indicator of congestion severity. By comparing the current traffic parameters with threshold values, the system was able to classify disruptions and simulate their impact accordingly. Only traffic records with a confidence score of 0.7 or higher were included to ensure data reliability. Any road closure or segment marked as blocked in the data was automatically treated as Heavy congestion, consistent with its impact on route accessibility (Li et al., 2024).

The classification criteria are summarized below:

Table 7

Classification of Disruption Levels

Level	Speed (km/h)	Jam Factor	Traversability	Interpretation
Light	> 20	≤ 3	open	Free-flow or minor slowdowns
Medium	5–20	4–6	open	Moderate congestion
Heavy	< 5 OR	> 6 OR	closed	Severe congestion or complete road closure

Rule-Based Incident Type Mapping Using HERE Flow Fields (for simulation):

Table 8

Rule-Based Incident Mapping

Simulated Incident Type	Rule-Based Condition (using Flow fields)	Explanation
Accident	speed < 2 km/h AND jamFactor > 7 AND traversability = open	Sudden slowdown with heavy congestion but road still open

Continuation of Table 8

Construction	speed drop ≥ 50% of freeFlow sustained over 30 min AND jamFactor < 7	Persistent slowdown likely due to long-term lane work
Congestion	jamFactor > 7 AND speed < 5 km/h	Classic vehicle buildup

Disabled Vehicle	speed ≈ 0 AND jamFactor < 4 AND short segment only	Blockage affecting a small segment only
Mass Transit Event	speed < 5 near known terminals at rush hour	Simulates crowding near stations (manual tagging)
Planned Event	Simulated slowdowns near known venues on weekends/evenings	Represents traffic around events like concerts
Road Hazard	speed drop > 60% AND jamTendency = +1 AND not closed	Debris, flooding, or potholes causing gradual jam
Road Closure	traversability = closed OR jamFactor = 10	Complete road segment closure
Weather	speed < 10 during time block, wide-area effect	Simulated monsoon or downpour scenarios
Lane Restriction	speed = 10–15 km/h AND jamTendency = +1 AND traversability = open	Simulates lane closures with limited access
Other	Random segment-level noise	Used to simulate unknown causes

Post-Simulation Phase: Graph Preparation for Evaluation

After traffic disruptions were injected into the network, the resulting traffic-aware graph served as input for both algorithms: the baseline DHL and the proposed Dynamic HC2L.

While both algorithms used the same graph with dynamic traffic weights and disruption annotations, they differed in how they responded to changes.

DHL computed labels once and did not adapt them during runtime. In contrast, Dynamic HC2L performed updates based on an Impact Score and a configurable threshold τ, determining whether to apply immediate or deferred (lazy) label recomputation.

A fixed set of origin–destination (OD) queries was used to evaluate routing performance. Resulting paths were compared against Google Maps using Fréchet Distance and Segment Overlap. Experiments were repeated across threshold values $\tau \in \{0.1, 0.2, ..., 1.0\}$, and results were recorded for labeling size, construction time, query response time, and route similarity.

Ethical Considerations

This research follows recognized ethical standards by using only public, nonpersonal data sources and simulated conditions. The datasets relied upon, which are OpenStreetMap, the HERE Traffic Flow API, and the Google Maps Directions API, do not include any personally identifiable information, individual mobility traces, or raw GPS logs. Routes retrieved from Google Maps serve only as examples of likely driving patterns and are accessed through standard public API requests, without collecting or storing data tied to any specific users.

Traffic scenarios were generated through a rule-based simulation process designed to reflect realistic disruptions without referencing live human activity or location tracking. All experiments took place in a controlled simulation environment, separate from any operational transportation systems. No changes were made to real-world infrastructure, and the work had no direct impact on roads, transit services, or travelers.

Synthetic user reports were incorporated purely to test how the algorithm responds to incident notifications, and these were created artificially rather than drawn from actual reports or devices. All data sources, APIs, and libraries are properly acknowledged, and results are presented transparently and in line with accepted academic practices for reproducibility and proper attribution.

Statistical Data Analysis

This section outlines the statistical methods used to analyze the performance metrics gathered from the experiments conducted in this study. The objective is to determine whether there are statistically significant differences between the Dynamic HC2L algorithm (with lazy updates and a threshold-based decision mechanism) and the DHL algorithm in terms of speed, memory usage, and path quality under simulated traffic conditions.

Evaluation Metrics

The following performance metrics were recorded and compared for both algorithm variants:

- Label Construction Time (in seconds)
- Labeling Size (in megabytes)
- Query Response Time (in microseconds)

Route Similarity Evaluation

To evaluate the realism of routes generated by the Dynamic HC2L algorithm, its computed paths were compared against reference routes from the Google Maps Directions API for identical origin-destination pairs in Quezon City. This comparison serves to address Statement of the Problem 3, which examines the alignment between the algorithm's outputs and established, real-world navigation solutions.

The evaluation will be conducted under both baseline (no disruption) and simulated disruption scenarios. This approach allows for an assessment of whether the Dynamic HC2L algorithm maintains its ability to produce geometrically and structurally plausible routes, even when faced with system disturbances.

To quantify the similarity, two complementary metrics were employed. The first, Fréchet Distance, assesses the geometric likeness between the two routes by measuring their spatial deviation. The second metric, Segment Overlap (%), offers a structural comparison by calculating the precise percentage of road segments that are shared between the algorithm's route and the Google Maps path. Together, these metrics aim to evaluate route realism, and not a direct performance comparison.

Threshold Sensitivity Testing

To determine the optimal threshold τ for update triggering, the system is evaluated across the range $T = \{0.1, 0.2, \ldots, 1.0\}$. For each τ, the following are recorded:

1. Number of label updates triggered
2. Average query response time
3. Labeling overhead
4. Route accuracy vs. Google Maps

Analysis includes line plots and variance metrics to assess trade-offs between update sensitivity and performance.

Statistical Method

Since each observation involves paired comparisons, where the same query or graph condition is processed by both the static and dynamic versions of the algorithm, a Paired Samples t-test is the appropriate statistical method for all metrics. This test determines whether the difference between two related samples

is statistically significant.

Paired Samples t-Test Formula

$$= \frac{\underline{d}}{s \sqrt{}} \quad t_{d/n} \tag{2}$$

Where:

- t = t-statistic (test value)
- $\underline{d}$ = mean of the differences between paired samples
- s_d = standard deviation of the differences
- n = number of paired observations **Supporting Formulas:**

Difference per pair:

$$d_i = x_i - y_i \tag{3}$$

Where x_i and y_i are the observed values for Dynamic and DHL respectively

Mean of differences:

$$\underline{d} = n^{-1} \sum_{i=1}^{n} \quad d_i \tag{4}$$

Standard Deviation of differences:

$$s_d = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} \quad (d_i - \underline{d})^2} \tag{5}$$

To ensure transparency and allow reproducibility, the following statistical values are also calculated:

Mean of each metric:

$$\underline{x} = n^{-1} \sum_{i=1}^{n} \quad x_i \tag{6}$$

Standard Deviation:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n} (x_i - \underline{x})^2} \quad (7)$$

Statistical Significance and Interpretation

The significance of results is determined using the p-value associated with the computed t-statistic. The following criteria are applied:

- If $p \leq 0.05$p: Reject the null hypothesis — a statistically significant difference exists.
- If $p > 0.05$**:** Fail to reject the null — no significant difference detected.

In the hypotheses, the null hypothesis assumes no significant difference between DHL and Dynamic HC2L. Each hypothesis is tested using the results from repeated trials across consistent inputs.

Table 9

Analytical Measures Used for Each Research Question

Research Question	Data Sources	Data Analysis
SOP1: What is the performance of the proposed Dynamic HC2L algorithm in terms of: a. Labeling time b. Labeling size c. Query response time	Dynamic HC2L performance across different trials	Paired Samples t-Test on time differences
SOP2: Is there a significant difference between the proposed Dynamic HC2L algorithm and the DHL algorithm in terms of: a. Labeling time b. Labeling size c. Query response time	Dynamic HC2L and DHL performance	Paired Samples tTest on label size differences

SOP3: What is the similarity between the routes produced by the proposed dynamic HC2L algorithm and real-world driver trajectories in Quezon City, measured by: a. Fréchet Distance b. Route segment overlap	Shortest path distances for 10 fixed query pairs in Dynamic HC2L and Google Maps	Paired Samples t-Test and % Deviation (optional)